

compilATOR

Documentación del Analizador Léxico

NASM x86-64 · Linux · Stage 1 of 6

Arquitectura general del compilador: el código fuente pasa por seis etapas antes de convertirse en un ejecutable. Este documento cubre únicamente la **Etapas 1: Análisis Léxico**. Las etapas restantes (Sintáctico, Semántico, Código Intermedio, Optimizador y Generador de Código) se documentarán conforme se implementen.

Índice

1. Estructura del Proyecto	3
2. main.asm — Entry Point	3
2.1. Constantes y declaraciones externas	3
2.2. Entry point y loop principal	4
2.3. Lookup en tabla de punteros	4
2.4. Syscall de escritura	5
3. symbol_table.asm — Datos Estáticos	5
3.1. Constantes de token ID	6
3.2. Strings de keywords	6
3.3. Strings de nombres de tokens	7
3.4. Tabla de punteros	7
4. lexer.asm — El Autómata Finito Determinista	7
4.1. Infraestructura de I/O	7
4.1.1. next_char	8
4.1.2. putback_char y append_char	9
4.2. get_token: el DFA	9
4.2.1. Clasificación del primer carácter	10
4.2.2. S1 — Identificador / Keyword	10
4.2.3. S2/S3/S4 — Número entero o flotante	11
4.2.4. S5/S6 — String literal	12
4.2.5. Operadores de dos caracteres	12
4.2.6. S22/S23/S24 — Comentarios de bloque	13
4.2.7. Macro para símbolos simples	13
4.3. Funciones auxiliares	14
4.3.1. is_alpha_or_under / is_digit	14
4.3.2. lookup_keyword	14
4.3.3. compare_str	15

5. Flujo Completo — Ejemplo	15
6. Próximas Etapas	16

Estructura del Proyecto

El analizador léxico está dividido en tres módulos ensamblados independientemente y unidos por el linker en un solo binario.

Archivo	Responsabilidad
symbol_table.asm	Datos estáticos: tokens, keywords, tabla de punteros
lexer.asm	Lógica del DFA: tokenización, lectura del input
main.asm	Entry point: loop de salida, syscalls de impresión

El flujo de ejecución es:

main.asm → llama `get_token` en `lexer.asm` → consulta datos en `symbol_table.asm`

Para compilar y ejecutar:

```
nasm -f elf64 symbol_table.asm -o symbol_table.o
nasm -f elf64 lexer.asm -o lexer.o
nasm -f elf64 main.asm -o main.o
ld symbol_table.o lexer.o main.o -o lexer
./lexer < test.src
```

O simplemente: `make run`

main.asm — Entry Point

Es el módulo más corto. Su única responsabilidad es coordinar el loop principal e imprimir los resultados. No sabe nada de cómo funciona el DFA ni de dónde vienen los datos.

Constantes y declaraciones externas

```
1 SYS_WRITE equ 1
2 SYS_EXIT equ 60
3 STDOUT equ 1
4 TK_EOF equ 36
5
6 extern get_token
7 extern lexeme
8 extern tk_name_table
9 extern str_arrow, str_close, str_header, str_tk_eof
```

Las constantes `equ` son sustituidas en tiempo de ensamblado — no ocupan memoria. `SYS_WRITE` y `SYS_EXIT` son los números de syscall que Linux x86-64 espera en `rax` para escribir en stdout y terminar el proceso respectivamente.

`extern` declara que esos símbolos existen en otro módulo. El linker resuelve las referencias al unir los tres `.o`. Sin `extern`, el ensamblador fallaría al no encontrar el símbolo.

Entry point y loop principal

```

1  global _start
2
3  _start:
4      mov rdi, str_header
5      call print_str
6
7  .token_loop:
8      call get_token          ; rax = token ID, lexeme[] =
9                               texto
10     cmp rax, TK_EOF
11     je  .at_eof
12
13     mov rdi, rax
14     call print_token_name    ; imprime "TK_INT"
15
16     mov rdi, str_arrow
17     call print_str          ; imprime " --> ["
18
19     mov rdi, lexeme
20     call print_str          ; imprime el lexema
21
22     mov rdi, str_close
23     call print_str          ; imprime "]\n"
24
25     jmp .token_loop

```

`_start` debe ser `global` para que el linker lo encuentre como punto de entrada del binario. El loop llama a `get_token` repetidamente hasta recibir `TK_EOF` (valor 36).

Las etiquetas con punto como `.token_loop` son **locales al procedimiento** — evitan colisiones de nombres entre módulos.

Cada iteración produce una línea como:

TK_INT ->[int]

Lookup en tabla de punteros

```

1  print_token_name:
2      dec rdi          ; token ID es 1-based, indice
3                        es 0-based
4      lea rsi, [tk_name_table]

```

```

4     mov rdi, [rsi + rdi*8] ; desreferenciar: obtener
      puntero al string
5     call print_str
6     ret

```

`tk_name_table` es un arreglo de 37 punteros de 8 bytes cada uno, definido en `symbol_table.asm`. La expresión `rdi*8` escala el índice por el tamaño de un puntero en 64 bits. El `mov` con corchetes **desreferencia** el puntero almacenado en la tabla, equivalente en C a:

```
char *name = tk_name_table[token_id - 1];
```

Syscall de escritura

```

1 print_str:
2     push rdi
3     xor rcx, rcx
4 .len_loop:
5     cmp byte [rdi + rcx], 0 ; buscar null terminator
6     je .write
7     inc rcx
8     jmp .len_loop
9 .write:
10    mov rdx, rcx ; longitud
11    mov rsi, rdi ; puntero al buffer
12    mov rdi, STDOUT ; file descriptor = 1
13    mov rax, SYS_WRITE ; syscall number = 1
14    syscall
15    pop rdi
16    ret

```

En assembly no hay `strlen` ni `printf` — la longitud se calcula manualmente recorriendo byte a byte hasta el 0 terminal. La syscall `SYS_WRITE` espera tres argumentos en registros específicos: `rdi` = file descriptor, `rsi` = buffer, `rdx` = longitud. El `push rdi` / `pop rdi` preserva el puntero original porque la syscall sobrescribe `rdi`.

Convención de llamada x86-64 (System V AMD64 ABI): los argumentos se pasan en orden por los registros `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`. El valor de retorno queda en `rax`. Los registros `rbx`, `rbp`, `r12–r15` deben ser preservados por el callee (callee-saved).

`symbol_table.asm` — Datos Estáticos

Este archivo no contiene ninguna instrucción ejecutable. Es la **fuentes de verdad** de todos los datos del compilador: cualquier módulo que necesite el nombre de un

token o comparar una keyword viene aquí vía [extern](#).

Constantes de token ID

```

1 TK_INT      equ 1
2 TK_FLOAT_KW equ 2
3 TK_BOOL     equ 3
4 ; ...
5 TK_EOF      equ 36
6 TK_ERROR    equ 37

```

[equ](#) es una sustitución de texto pura — el ensamblador reemplaza cada aparición de [TK_INT](#) por [1](#) antes de generar código. No ocupa ningún byte en el binario final.

La tabla completa tiene 37 tokens:

ID	Token	ID	Token	ID	Token
1	TK_INT	14	TK_LIT_STRING	27	TK_AND
2	TK_FLOAT	15	TK_ID	28	TK_OR
3	TK_BOOL	16	TK_ASSIGN	29	TK_NOT
4	TK_STRING	17	TK_PLUS	30	TK_LPAREN
5	TK_IF	18	TK_MINUS	31	TK_RPAREN
6	TK_ELSE	19	TK_STAR	32	TK_LBRACE
7	TK_WHILE	20	TK_SLASH	33	TK_RBRACE
8	TK_FOR	21	TK_EQ	34	TK_SEMICOLON
9	TK_RETURN	22	TK_NEQ	35	TK_COMMA
10	TK_TRUE	23	TK_LT	36	TK_EOF
11	TK_FALSE	24	TK_GT	37	TK_ERROR
12	TK_LIT_INT	25	TK_LTE		
13	TK_LIT_FLOAT	26	TK_GTE		

Strings de keywords

```

1 global kw_int, kw_float, kw_bool, kw_string
2 global kw_if, kw_else, kw_while, kw_for, kw_return
3 global kw_true, kw_false
4
5 kw_int      db 'int', 0
6 kw_float    db 'float', 0
7 kw_bool     db 'bool', 0
8 ; ...

```

[db](#) (*define byte*) reserva bytes en la sección [.data](#). ['int'](#) son tres bytes ASCII seguidos de un [0](#) terminal. [lexer.asm](#) los importa con [extern](#) para usarlos en el lookup de palabras reservadas.

Strings de nombres de tokens

```

1 strTk_int      db 'TK_INT          ', 0
2 strTk_float    db 'TK_FLOAT        ', 0
3 ; ...

```

Todos tienen el mismo ancho (padding de espacios) para que la salida quede alineada en columnas. La flecha `->` siempre aparece en la misma posición horizontal.

Tabla de punteros

```

1 global tk_name_table
2
3 tk_name_table:
4     dq strTk_int      ; TK_INT      = 1
5     dq strTk_float_kw ; TK_FLOAT    = 2
6     ; ... 37 entradas en total
7     dq strTk_error    ; TK_ERROR    = 37

```

`dq` (*define quadword*) reserva 8 bytes — el tamaño exacto de un puntero en 64 bits. La tabla es equivalente en C a:

```

const char *tk_name_table[] = {
    "TK_INT",          /* index 0, token ID 1 */
    "TK_FLOAT",        /* index 1, token ID 2 */
    /* ... */
};

```

`main.asm` indexa la tabla con `[tk_name_table + (id-1)*8]` para obtener el puntero al string del nombre correspondiente.

lexer.asm — El Autómata Finito Determinista

Es el módulo central. Implementa un DFA de 27 estados que lee caracteres del input y los agrupa en tokens. Exporta `getToken`, `lexeme` y `lexeme_len`. Importa los strings de keywords de `symbol_table.asm`.

Infraestructura de I/O

El `lexer` no lee un carácter a la vez desde el sistema operativo — eso sería una `syscall` por carácter, lo cual es extremadamente lento. En cambio, usa un **buffer de 4096 bytes**:

```

1 BUF_SIZE equ 4096

```

```

2 LEXEME_MAX equ 256
3
4 section .bss
5 buf          resb BUF_SIZE      ; buffer de lectura
6 buf_pos      resq 1             ; posicion actual en el buffer
7 buf_len      resq 1             ; bytes validos en el buffer
8 putback      resb 1             ; registro de retroceso (
    lookahead)
9 has_putback   resb 1             ; flag: hay caracter en putback
    ?
10 lexeme       resb LEXEME_MAX    ; texto del token actual
11 lexeme_len    resq 1             ; longitud del token actual

```

`resb` y `resq` reservan bytes sin inicializar en `.bss` (a diferencia de `db/dq` que van en `.data` con valores iniciales). El SO inicializa `.bss` a cero al cargar el proceso.

next_char

```

1 next_char:
2     cmp byte [has_putback], 1    ; hay caracter en espera?
3     jne .from_buffer
4     mov byte [has_putback], 0
5     mov al, [putback]
6     ret
7
8 .from_buffer:
9     mov rcx, [buf_pos]
10    cmp rcx, [buf_len]
11    jl  .read_from_buf
12
13    ; Buffer vacio: rellenar con syscall read
14    mov rax, SYS_READ
15    mov rdi, STDIN
16    mov rsi, buf
17    mov rdx, BUF_SIZE
18    syscall
19    cmp rax, 0
20    jle .eof
21    mov qword [buf_len], rax
22    mov qword [buf_pos], 0
23
24 .read_from_buf:
25    mov rcx, [buf_pos]
26    mov al, [buf + rcx]
27    inc rcx
28    mov [buf_pos], rcx
29    ret
30
31 .eof:

```



```

32     xor al, al
33     ret

```

`next_char` retorna el siguiente carácter en `al`. Primero verifica el registro de putback (lookahead de 1 carácter). Si el buffer está vacío, llama a `SYS_READ` para leer hasta 4096 bytes. Retorna `0` en EOF.

putback_char y append_char

```

1  putback_char:
2      mov [putback], al      ; guardar character
3      mov byte [has_putback], 1
4      ret
5
6  append_char:
7      mov rcx, [lexeme_len]
8      cmp rcx, LEXEME_MAX - 1
9      jge .full              ; no desborden el buffer
10     mov [lexeme + rcx], al
11     inc rcx
12     mov [lexeme_len], rcx
13     mov byte [lexeme + rcx], 0 ; null terminator siempre
                                   actualizado
14 .full:
15     ret

```

`putback_char` implementa el **lookahead de 1 carácter**: cuando el DFA lee un carácter de más para decidir si un operador es simple o doble (e.g. `<` vs `<=`), devuelve el carácter al stream para que la próxima llamada lo vuelva a leer.

`append_char` agrega `al` al lexema actual, manteniendo siempre el null terminator actualizado. Respeta el límite de `LEXEME_MAX - 1` bytes para evitar desbordamiento.

get_token: el DFA

```

1  get_token:
2      push rbp
3      mov rbp, rsp
4
5      ; Limpiar lexeme
6      mov qword [lexeme_len], 0
7      mov byte [lexeme], 0
8
9  .s0_skip_ws:
10     call next_char
11     cmp al, 0
12     je .emit_eof

```

```

13     cmp al, ' '
14     je  .s0_skip_ws
15     cmp al, 9      ; tab
16     je  .s0_skip_ws
17     cmp al, 10     ; newline
18     je  .s0_skip_ws
19     cmp al, 13     ; carriage return
20     je  .s0_skip_ws

```

El Estado S0 ignora todo el whitespace al inicio de cada token. Al encontrar un carácter no-blanco, entra a la clasificación.

Clasificación del primer carácter

```

1     call is_alpha_or_under ; [a-zA-Z_] ?
2     jc  .state_id
3
4     call is_digit         ; [0-9] ?
5     jc  .state_number
6
7     cmp al, '"'
8     je  .state_string
9
10    cmp al, '='
11    je  .state_eq
12    cmp al, '!'
13    je  .state_not
14    cmp al, '<'
15    je  .state_lt
16    cmp al, '>'
17    je  .state_gt
18    cmp al, '&'
19    je  .state_amp
20    cmp al, '|'
21    je  .state_pipe
22    cmp al, '/'
23    je  .state_slash
24    ; ... simbolos simples ...

```

Las funciones `is_alpha_or_under` e `is_digit` retornan el resultado en el **Carry Flag** (CF): CF=1 si la condición se cumple, CF=0 si no. Esto permite usar `jc` (*jump if carry*) como branch, sin necesidad de comparar el valor de retorno en un registro.

S1 — Identificador / Keyword

```

1 .state_id:
2     call append_char

```

```

3  .id_loop:
4      call next_char
5      cmp al, 0
6      je .id_end
7      call is_alnum_or_under
8      jc .id_loop_continue
9      call putback_char      ; caracter no pertenece al id
10     jmp .id_end
11 .id_loop_continue:
12     call append_char
13     jmp .id_loop
14 .id_end:
15     call lookup_keyword      ; rax = TK_* o TK_ID
16     jmp .done

```

Lee caracteres mientras sean alfanuméricos o guión bajo. Al terminar, `lookup_keyword` compara el lexema acumulado contra las 11 palabras reservadas. Si coincide, retorna el token de keyword; si no, retorna `TK_ID`.

S2/S3/S4 — Número entero o flotante

```

1  .state_number:
2      call append_char
3  .num_int_loop:
4      call next_char
5      cmp al, 0
6      je .num_is_int
7      call is_digit
8      jc .num_int_digit
9      cmp al, '.'
10     je .num_dot      ; posible flotante
11     call putback_char
12     jmp .num_is_int
13 .num_int_digit:
14     call append_char
15     jmp .num_int_loop
16 .num_dot:
17     call append_char      ; guardar '.'
18     call next_char
19     call is_digit
20     jc .num_float_first_digit
21     call putback_char      ; "5.x" -> flotante sin fraccion
22     jmp .num_is_float

```

La distinción entre entero y flotante se hace al encontrar un punto decimal. El DFA pasa de S2 (entero) a S3 (flotante) y continúa consumiendo dígitos. Usa `putback` para no perder el carácter que determinó el fin del número.

S5/S6 — String literal

```

1  .state_string:
2      ; La comilla de apertura NO se guarda en el lexema
3  .str_loop:
4      call next_char
5      cmp al, 0
6      je .str_terminated      ; EOF sin cerrar -> error
7      cmp al, '"'
8      je .str_closed
9      cmp al, 10              ; newline sin cerrar -> error
10     je .str_terminated
11     call append_char
12     jmp .str_loop
13 .str_closed:
14     mov rax, TK_LIT_STRING
15     jmp .done

```

Las comillas delimitadoras no se incluyen en el lexema — solo el contenido del string. Un EOF o newline sin cerrar la cadena produce [TK_ERROR](#).

Operadores de dos caracteres

El patrón es el mismo para todos. Se muestra el caso de = / ==:

```

1  .state_eq:
2      call next_char
3      cmp al, '='
4      jne .eq_is_assign
5      ; Es '=='
6      mov byte [lexeme], '='
7      mov byte [lexeme+1], '='
8      mov byte [lexeme+2], 0
9      mov qword [lexeme_len], 2
10     mov rax, TK_EQ
11     jmp .done
12 .eq_is_assign:
13     call putback_char      ; devolver caracter leído de
14                             mas
15     mov byte [lexeme], '='
16     mov byte [lexeme+1], 0
17     mov qword [lexeme_len], 1
18     mov rax, TK_ASSIGN
19     jmp .done

```

Al leer =, el DFA consume un carácter más. Si es otro =, emite [TK_EQ](#). Si no, lo devuelve con [putback_char](#) y emite [TK_ASSIGN](#). El mismo patrón aplica para !=, <=, >=, && y ||.

S22/S23/S24 — Comentarios de bloque

```

1  .state_slash:
2      call next_char
3      cmp al, '*'
4      je .comment_start
5      call putback_char      ; es solo '/'
6      mov rax, TK_SLASH
7      jmp .done
8
9  .comment_start:
10 .cmt_loop:
11     call next_char
12     cmp al, 0
13     je .cmt_eof_error
14     cmp al, '*'
15     jne .cmt_loop
16 .cmt_after_star:
17     call next_char
18     cmp al, '/'
19     je .cmt_closed
20     cmp al, '*'
21     je .cmt_after_star      ; maneja '***/' correctamente
22     jmp .cmt_loop
23 .cmt_closed:
24     ; Comentario consumido: tail-call a get_token
25     mov qword [lexeme_len], 0
26     mov byte [lexeme], 0
27     pop rbp
28     jmp get_token

```

Los comentarios `/* */` se descartan completamente. Al cerrarse el comentario, en lugar de retornar (lo que emitiría un token vacío), el código hace un **tail-call**: restaura el stack frame y salta directamente a `get_token` para buscar el siguiente token real. Esto evita crear un frame de stack innecesario y es más eficiente que una llamada recursiva.

Macro para símbolos simples

```

1  %macro single_tok 2      ; %1 = char ASCII, %2 = token ID
2      mov byte [lexeme], %1
3      mov byte [lexeme+1], 0
4      mov qword [lexeme_len], 1
5      mov rax, %2
6      jmp .done
7  %endmacro
8
9  .single_plus:    single_tok '+', TK_PLUS

```

```
10 .single_minus:  single_tok '-', TK_MINUS
11 .single_star:   single_tok '*', TK_STAR
12 ; ...
```

`%macro` define una macro NASM parametrizada. Cada `single_tok` se expande a 4 instrucciones inline evitando duplicar código para los 9 símbolos de un solo carácter.

Funciones auxiliares

`is_alpha_or_under / is_digit`

```
1  is_alpha_or_under:
2      cmp al, '_'
3      je .yes
4      cmp al, 'a'
5      jl .check_upper
6      cmp al, 'z'
7      jle .yes
8  .check_upper:
9      cmp al, 'A'
10     jl .no
11     cmp al, 'Z'
12     jle .yes
13 .no:  clc
14     ret
15 .yes: stc
16     ret
```

Retornan el resultado en el Carry Flag en lugar de en un registro de propósito general. Esto permite usar `jc` / `jnc` directamente en el sitio de llamada sin instrucciones adicionales de comparación, haciendo el código más compacto.

`lookup_keyword`

```
1  lookup_keyword:
2      mov rsi, kw_int
3      call compare_str
4      jc .kw_int
5      mov rsi, kw_float
6      call compare_str
7      jc .kw_float
8      ; ... 11 keywords en total ...
9      mov rax, TK_ID
10     ret
11
12 .kw_int:  mov rax, TK_INT      ; ret
13         ret
```

```
14 ; ...
```

Compara el lexema contra cada keyword secuencialmente. En el peor caso (identificador que no es keyword) hace 11 comparaciones. Para el tamaño del lenguaje actual esto es aceptable. Una tabla hash sería más eficiente con un vocabulario mayor.

compare_str

```
1  compare_str:
2      push rdi
3      push rsi
4      push rcx
5      mov  rdi, lexeme
6      xor  rcx, rcx
7  .loop:
8      mov  al, [rdi + rcx]
9      mov  bl, [rsi + rcx]
10     cmp  al, bl
11     jne  .no
12     cmp  al, 0          ; ambos cero = iguales
13     je   .yes
14     inc  rcx
15     jmp  .loop
16 .yes:
17     pop  rcx
18     pop  rsi
19     pop  rdi
20     stc
21     ret
22 .no:
23     pop  rcx
24     pop  rsi
25     pop  rdi
26     clc
27     ret
```

Comparación byte a byte de dos strings null-terminated. Preserva `rdi`, `rsi` y `rcx` en el stack porque son registros caller-saved que los sitios de llamada podrían estar usando. Retorna `CF=1` si iguales.

Flujo Completo — Ejemplo

Para el input `int x = 42;`, el flujo es:

1. `_start` llama `print_str` con el header y entra al loop.
2. `get_token` entra a `S0`, lee `'i'`.

3. `is_alpha_or_under` retorna `CF=1` → va a `.state_id`.
4. Acumula `'i'`, `'n'`, `'t'` en `lexeme[]`.
5. Lee `' '` → no es alfanumérico → putback del espacio.
6. `lookup_keyword` compara contra `kw_int` → match → retorna `TK_INT` en `rax`.
7. `main.asm` imprime `TK_INT` → `[int]`.
8. Segunda iteración: `S0` consume el espacio, lee `'x'`.
9. `.state_id` → `lookup_keyword` no encuentra match → retorna `TK_ID`.
10. `main.asm` imprime `TK_ID` → `[x]`.
11. ...y así sucesivamente hasta `TK_EOF`.

Próximas Etapas

Etapas	Módulo	Descripción
2	<code>parser.asm</code>	Analizador sintáctico — parser descendente recursivo, construye el AST en memoria
3	<code>semantic.asm</code>	Analizador semántico — type checking, tabla de símbolos dinámica con scopes
4	<code>irgen.asm</code>	Generador de código intermedio — emite TAC (three-address code)
5	<code>optimizer.asm</code>	Optimizador — constant folding, dead code elimination
6	<code>codegen.asm</code>	Generador de código — emite x86-64 NASM directamente